> LMSC_261

Lecture 07: Python Classes

Fall 2026 • Introduction to Computer Programming

E.K. Ellington Scott, Ph.D.

Berklee

Overview

- PythPython review
- Why classes and objects?
- Attributes, methods, __init__, and self
- Encapsulation, composition, and inheritance
- Connections to other languages

Warm Up Check

What does this program print?

```
tracks = ["drums", "bass", "keys"]
for i, track in enumerate(tracks):
    print(f"{i + 1}: {track}")
```

Think → predict → run it

Review: Variables + Objects + Data Types

int	float	str	bool
42	0.75	"Berklee"	True

```
tempo = 120
artist = "Esperanza Spalding"
is_live = True
```

Review: Lists

LIST EXAMPLE

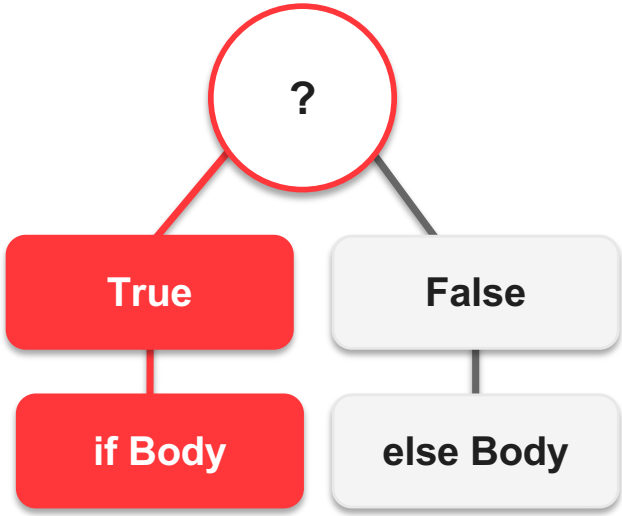
```
tracks = ["kick", "snare", "hat"]
```

kick	snare	hat
0	1	2

A list stores multiple values in order. Use an index (starting at 0) to access an item. Lists can be changed after creation.

Review: Conditionals

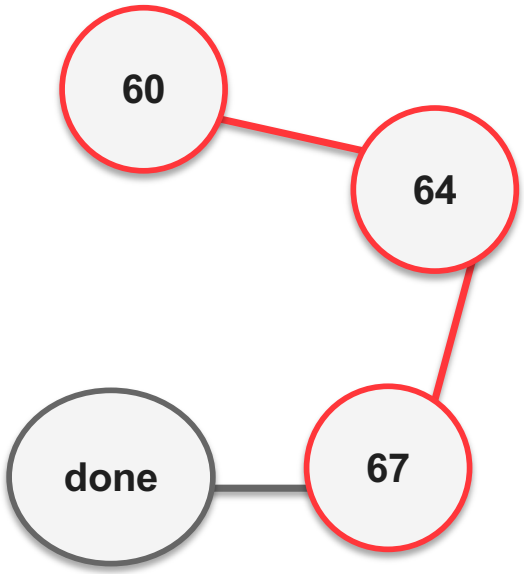
```
if velocity > 100:
    print("accent")
elif velocity > 0:
    print("normal note")
else:
    print("silence")
```



Review: Loops

```
notes = [60, 64, 67]

for note in notes:
    print(notes)
```

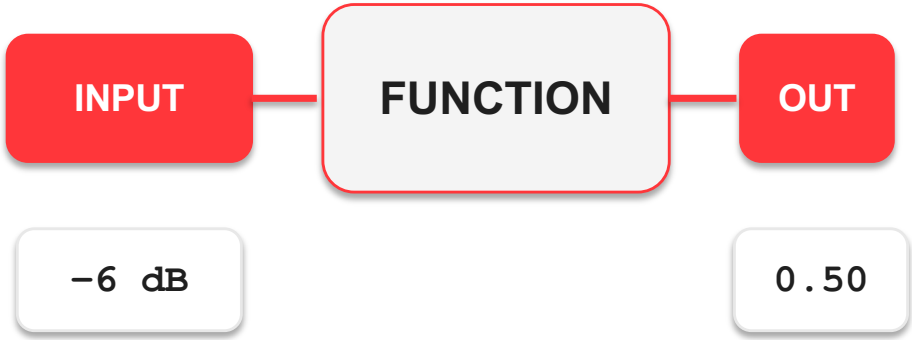


A loop repeats a block of code over a sequence or while a condition is true.

Review: Functions

```
def db_to_gain(db):
    gain = 10 ** (db / 20)
    return gain

x = db_to_gain(-6)
```



Functions package behavior

Review: Scope + Mutation

Local variable

```
def set_level():
    level = 0.8    # local
    return level
```

Mutable object

```
levels = [0.5, 0.8]
levels.append(1.0)
# the list changed
```

Review: Modules + Imports




```
import math
from pathlib import Path
import py5
```

OOP: The Big Picture

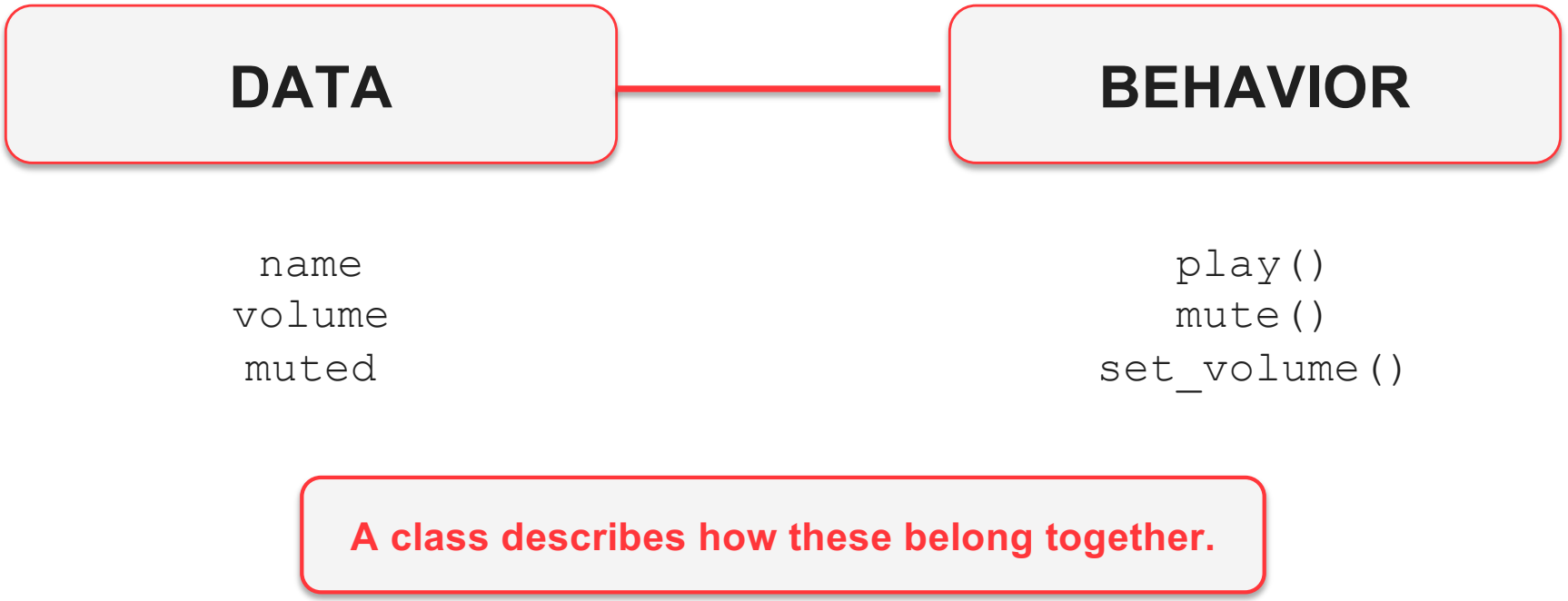
- OOP stands for **object-oriented programming**.
- It organizes code around objects that bundle data and behavior together.
- A class is a blueprint. An object is one specific instance built from that blueprint.
- OOP is useful when your program has clear “things” to model, like tracks, instruments, playlists, or students.

Core Ideas in OOP

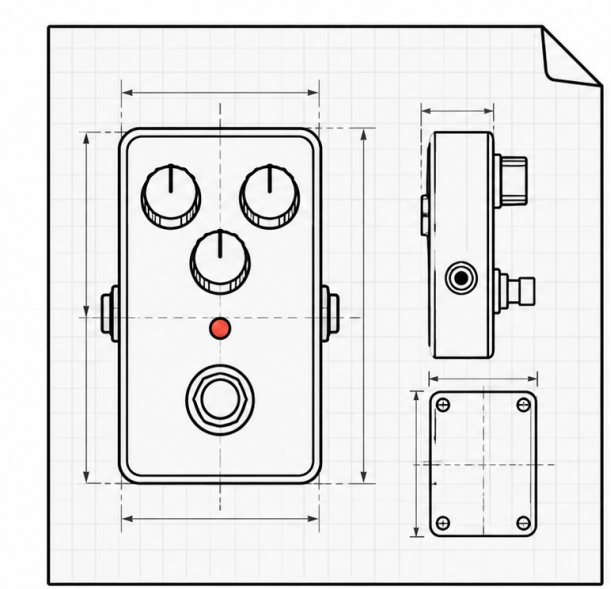
- **Encapsulation**: keep related data and behavior together in one place.
- **Abstraction**: show what an object does without exposing every detail.
- **Inheritance**: make a new class by extending an existing one.
- **Polymorphism**: different objects can respond to the same method name in different ways.

Classes + Objects

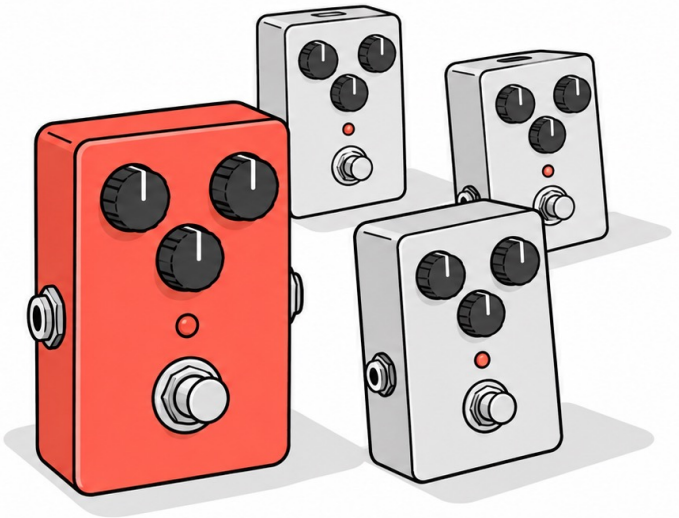
The Core Idea: Data + Behavior



Class = Blueprint • Object = One Instance



CLASS = blueprint



OBJECTS = pedals

Vocabulary: Class, Object, Attribute, Method



Your First Class

```
class Track:
    def __init__(self, name, volume):
        self.name = name
        self.volume = volume
```

class Track: defines a new type

__init__ runs when a new object is created

__init__: Build the Starting State

```
class Track:
    def __init__(self, name, volume):
        self.name = name
        self.volume = volume
        self.muted = False
```

Think: setup() for an object

name	"Drums"
volume	0.8
muted	False

What Is self?

self means: “this particular object”

drums

```
self.volume =
0.8
```

bass

```
self.volume =
0.6
```

keys

```
self.volume =
0.9
```

Same class. Different self. Different state.

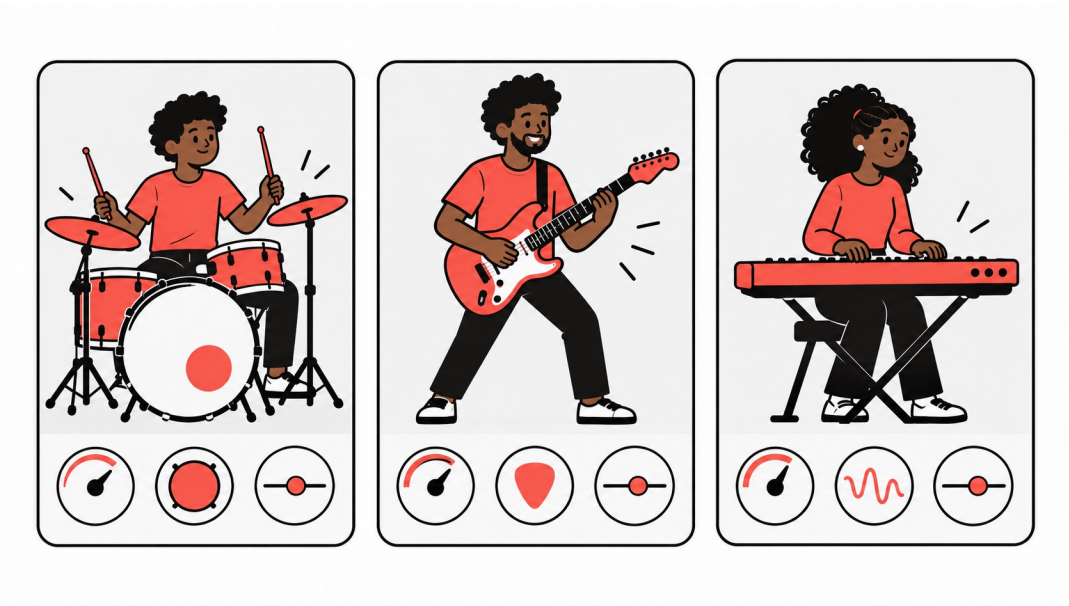
Creating Objects

```
drums = Track("Drums", 0.8)
bass = Track("Bass", 0.6)
keys = Track("Keys", 0.9)

print(drums.name)
print(bass.volume)
```



Attributes Hold Object State

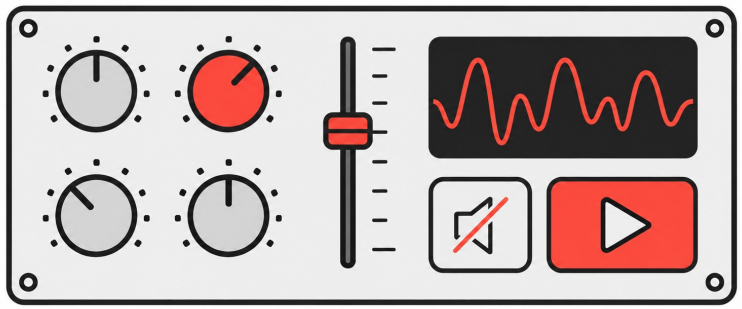


Each object can store different values even though all use the same class.

Methods Give Objects Behavior

```
class Track:
    ...
    def mute(self):
        self.muted = True

    def play(self):
        print(self.name)
```



Methods are functions inside a class.

Methods Can Change State

```
class Track:
    def __init__(self, name, volume):
        self.name = name
        self.volume = volume

    def turn_down(self, amount):
        self.volume -= amount
```




```
drums.turn_down(0.2)
```

Exercise 1: Build a Track Class

- Create a Track class
- Attributes: name, instrument, volume
- Method: describe() prints a summary
- Method: turn_down(amount) lowers volume
- Create at least two Track objects

```
class Track:
    def __init__(self, ...):
        ...

    def describe(self):
        ...

    def turn_down(self, amount):
        ...
```

Test every method!

Exercise 1: One Possible Solution

```
class Track:
    def __init__(self, name, instrument, volume):
        self.name = name
        self.instrument = instrument
        self.volume = volume

    def describe(self):
        print(f"{self.name}: {self.instrument} @ {self.volume}")

    def turn_down(self, amount):
        self.volume -= amount
```

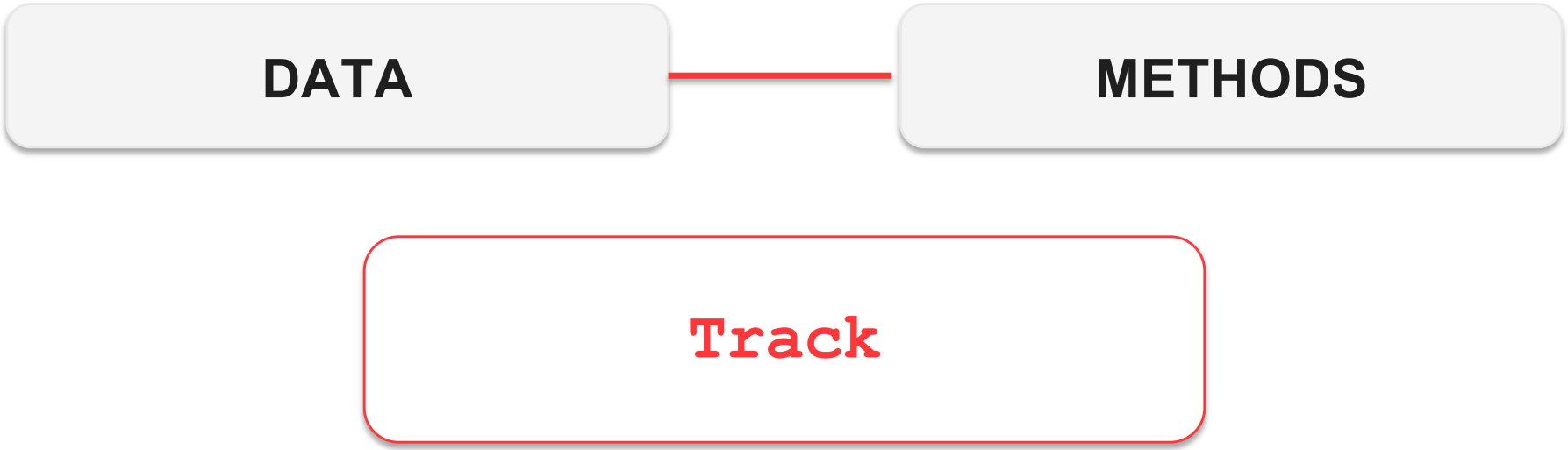
```
drums = Track("Beat", "Drums", 0.8)
bass = Track("Low End", "Bass", 0.7)

drums.turn_down(0.2)
drums.describe()
```

State + behavior now travel together.

Building Better Classes

Encapsulation: Keep Related Things Together



Encapsulation makes it easier to reason about one object at a time.

Instance Attributes vs. Class Attributes

```
class Track:
    # class attribute
    sample_rate = 48000

    def __init__(self, name):
        # instance attribute
        self.name =
```

Shared by every Track

```
Track.sample_rate
→ 48000
```

Unique to each object

```
drums.name ≠ bass.name
```

__str__: Make Objects Human Readable

```
class Track:
    ...
    def __str__(self):
        return f"{self.name} ({self.volume:.2f})"

drums = Track("Drums", 0.8)
print(drums)
```

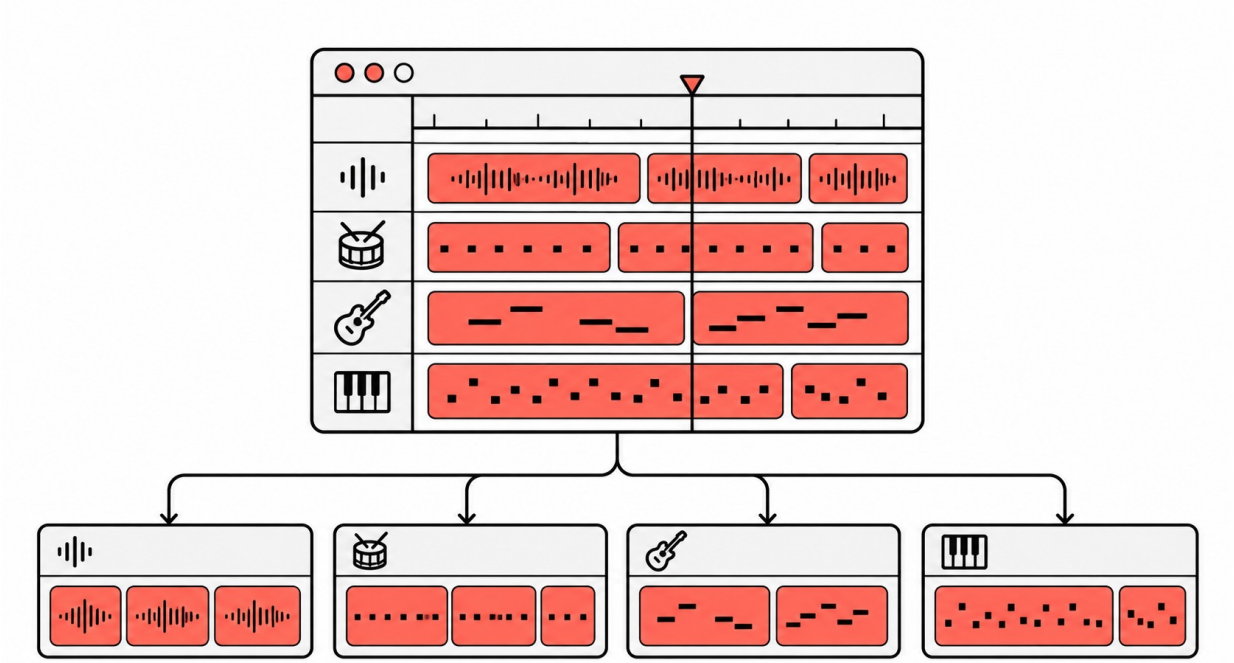
Instead of...

```
<__main__.Track object ...>
```

You get...

```
Drums (0.80)
```

Composition: Objects Can Contain Other Objects



A Song can contain Track objects. This is a “has a” relationship.

Composition in Code

```
class Song:
    def __init__(self, title):
        self.title = title
        self.tracks = []

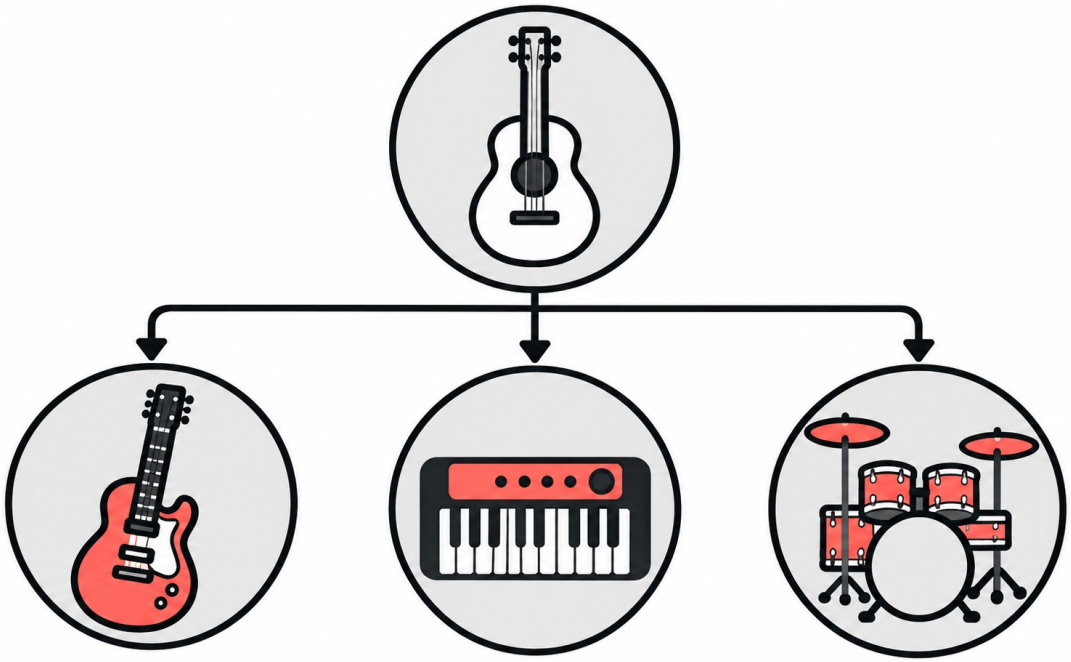
    def add_track(self, track):
        self.tracks.append(track)
```

```
song = Song("Demo")
song.add_track(drums)
song.add_track(bass)

print(song.tracks)
```

Song has Tracks

Inheritance: One Class Can Build on Another



Inheritance represents an “is a” relationship.

Inheritance in Code

```
class Instrument:
    def __init__(self, name):
        self.name = name

    def play(self):
        print("Playing")
```

```
class Synth(Instrument):
    def __init__(self, name, preset):
        super().__init__(name)
        self.preset = preset

    def play(self):
        print(f"Synth: {self.preset}")
```

Polymorphism: Same Method Name, Different Behavior

```
instruments = [
    Synth("Keys", "Warm Pad"),
    DrumMachine("Beat", "808")
]

for instrument in instruments:
    instrument.play()
```

Same message

`instrument.play()`

Different object
→ different result

Inheritance or Composition?

Use inheritance for “is a”

Synth is an Instrument

Good for shared behavior

Use composition for “has a”

Song has Tracks

Often simpler and more flexible

Do not use inheritance just to avoid typing code.

Common Class Mistakes

Missing self

```
def play():
```

Using class attr by accident

```
shared_list = []
```

Forgetting ()

```
track.play
```

Confusing class + object

```
Track.name vs drums.name
```

When Should I Use a Class?

- You have many values that describe one thing
- That thing has behavior
- You need many similar objects
- The object has state that changes over time

Good examples

- **Track**
- **Instrument**
- **Playlist**
- **MIDI Note**
- **Game Character**

Exercise 2: Instrument Family

- Create an Instrument parent class
- Give it name and volume attributes
- Give it a play() method
- Create two child classes
- Override play() in each child
- Put the objects in a list and loop over play()

```
class Instrument:
    ...

class Synth(Instrument):
    ...

class DrumMachine(Instrument):
    ...
```

Exercise 2: One Possible Solution

```
class Instrument:
    def __init__(self, name, volume):
        self.name = name
        self.volume = volume

    def play(self):
        print(f"Playing {self.name}")

class Synth(Instrument):
    def play(self):
        print(f"{self.name}: warm synth chord")
```

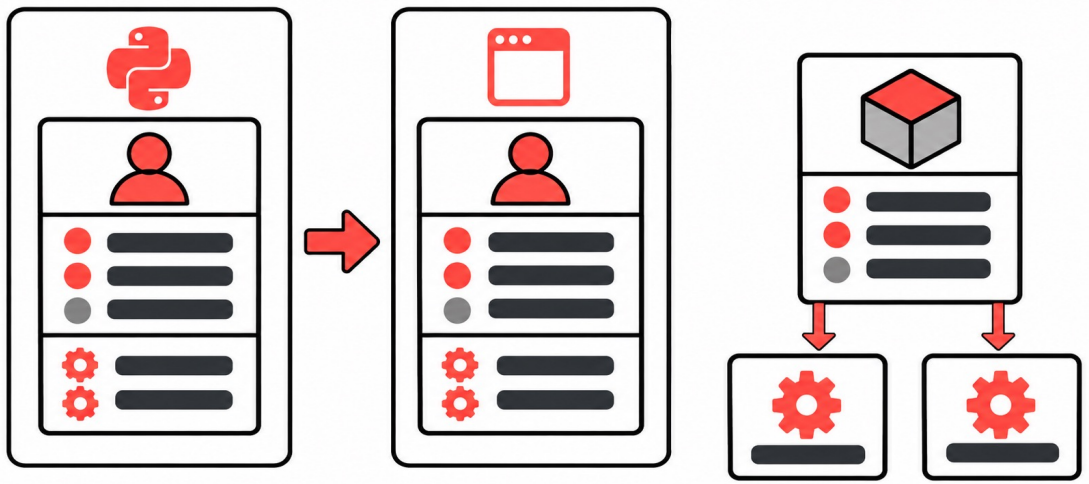
```
class DrumMachine(Instrument):
    def play(self):
        print(f"{self.name}: boom ba

instruments = [
    Synth("Keys", 0.8),
    DrumMachine("Beat", 0.9)
]

for item in instruments:
    item.play()
```

Language Bridges

The Same Idea Across Languages



The syntax changes. The design idea survives.

Python → JavaScript

Python

```
class Track:
    def __init__(self, name):
        self.name = name

    def play(self):
        print(self.name)
```

JavaScript

```
class Track {
  constructor(name) {
    this.name = name;
  }

  play() {
    console.log(this.name);
  }
}
```

`self` → `this` • `__init__` → `constructor`

Python → C: Same Thinking, Different Tools

Important: C does not have classes built into the language.

```python
class Track:
    def __init__(self, volume):
        self.volume = volume

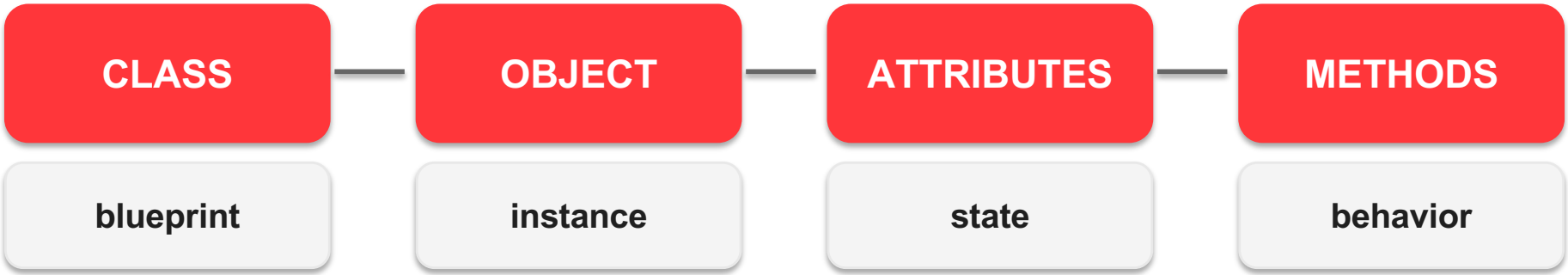
    def mute(self):
        self.volume = 0
```

```c
typedef struct {
    float volume;
} Track;

void mute(Track *track) {
    track->volume = 0.0f;
}
```

C often separates the data structure from the functions that operate on it.

Final Mental Model



Classes let us model programs as reusable things with data and behavior.

Before You Leave: Can You Explain...?

- What is the difference between a class and an object?
- What does self refer to?
- What does __init__ do?
- When would you use composition?

Questions?